

# Descripción de XML

## Introducción

XML (*eXtensible Markup Language*) es un lenguaje de etiquetas o un lenguaje de marcado que **estructura** y guarda de forma ordenada la **información**. No representa datos por sí mismo, solamente organiza la estructura. En XML, las etiquetas son creadas por el programador.

XML **ahorra tiempos de desarrollo** y proporciona **ventajas**, dotando a webs y a aplicaciones de una forma realmente potente de guardar la información. Además, se ha convertido en un formato universal que ha sido asimilado por todo tipo de sistemas operativos y dispositivos móviles.

**Para ampliar conocimientos sobre XML:**

- [Tutorial de XML](#) de W3C Schools
- [Introducción a XML](#) de Mozilla Developer Network (MDN)

## Características del documento XML

Un documento XML es un **documento de texto** que tiene las siguientes características:

- La extensión de fichero es `.xml`.
- Está compuesto de parejas de etiquetas estructuradas en árbol, que describen una función en la organización del documento.
- Puede editarse con cualquier editor de texto.
- Es interpretado por los navegadores web.

## Características del lenguaje XML

Las características básicas de XML son:

- Dado que XML se concibió para trabajar en la web, es directamente compatible con protocolos que ya funcionan (como HTTP y HTTPS).
- Todo documento que verifique las reglas de XML está conforme con SGML.
- No se requieren conocimientos de programación para realizar tareas sencillas en XML.
- Los documentos XML son fáciles de crear.

- La difusión de los documentos XML está asegurada, ya que cualquier procesador de XML puede leer un documento de XML.
- El marcado de XML es legible para los humanos (*human readable*).
- El diseño XML es formal y conciso.
- XML es extensible, adaptable y aplicable a una gran variedad de situaciones.
- XML es orientado a objetos.
- Todo documento XML se compone exclusivamente de datos de marcado y datos carácter entremezclados.

El **proceso de creación** de un documento XML pasa por varias etapas, en las que el éxito de cada una de ellas se basa en la calidad de la anterior. Estas etapas son:

- Especificación de requisitos.
- Diseño de etiquetas.
- Marcado de los documentos.

## Marcado

El **marcado** en XML son etiquetas que se añaden a un texto para estructurar el contenido del documento. Esta información extra permite a los ordenadores *interpretar* los textos. El marcado es todo lo que se sitúa entre:

- `<` y `>` en el caso de elementos.
- `&` y `;` en el caso de entidades.

Más adelante veremos qué son los elementos y las entidades.

### EJEMPLO DE MARCADO: ELEMENTO

```
<body>
```

### EJEMPLO DE MARCADO: ENTIDAD

```
&gt;
```

Los datos que se encuentran **encerrados entre marcas** son los que forman la **verdadera información** del documento XML.

Por ejemplo, en el siguiente documento, el **dato** es `12.39`:

```
<precio>12.39</precio>
```

El marcado puede ser tan rico como se quiera. Puede ser interesante detectar necesidades futuras y crear documentos con una estructura fácilmente actualizables

## Comentarios

Los documentos XML pueden tener **comentarios**, que se utilizan para incluir información para el desarrollador, ya que éstos **no son interpretados** por el interprete XML.

Se incluyen entre las cadenas `<!--` y `-->`.

```
<!-- Ejemplo de comentario -->
```

Pueden estar en cualquier posición en el documento excepto:

- Antes del prólogo.
- Dentro de una etiqueta.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Comentario correcto -->
<b>
  <c cantidad="4">cccc<!-- Comentario correcto --></c>
  <d cantidad="2">dd</d><!-- Comentario correcto -->
</b>
```

```
<!-- Comentario incorrecto -->
<?xml version="1.0" encoding="UTF-8"?>

<b>
  <c cantidad="4" <!-- Comentario incorrecto -->>cccc</c>
  <d cantidad="2">dd</d>
</b>
```

Los comentarios no pueden contener dos guiones medios seguidos (`--`).

```
<!-- Comentario incorrecto -- (contiene dos guiones medios seguidos
dentro del comentario) -->
```

# Estructura de un documento XML

Los documentos XML están formados por las siguientes partes:

- Un **prólogo** (opcional)
- Un **ejemplar** (obligatorio)

En el siguiente ejemplo, las dos primeras líneas se corresponden con el prólogo, mientras que el resto se corresponden con el ejemplar:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
  <isbn>978-2-7460-4958-1</isbn>
  <edicion>1</edicion>
  <paginas>347</paginas>
</libro>
```

## Prólogo

El prólogo es una parte **opcional** de un documento XML que, si se incluye, debe preceder al ejemplar. Su inclusión facilita el procesamiento de la información del ejemplar.

El **prólogo** está dividido en dos partes:

- La declaración XML.
- La declaración del tipo de documento.

## Declaración XML

En el caso de incluirse, ha de ser **la primera línea del documento**. De no ser así, se genera un error que impide que el documento sea procesado. El hecho de que sea opcional, permite el procesamiento de documentos HTML y SGML como si fueran XML. Si fuera obligatoria, éstos deberían incluir una declaración de versión XML que no tienen.

El prólogo puede tener tres funciones:

1. Declaración la versión de XML usada para elaborar el documento.

2. Declaración de la codificación empleada para representar los caracteres.
3. Declaración de la autonomía del documento.

De base, una declaración XML tiene el siguiente aspecto:

```
<?xml ?>
```

## Versión de XML

La declaración de la versión de XML usada para elaborar el documento se indica con el atributo `version`:

```
<?xml version="1.0" ?>
```

En este caso, se indica que el documento fue creado para la versión `1.0` de XML.

## Codificación de caracteres

La declaración de la codificación empleada para representar los caracteres se indica con el atributo `encoding`:

```
<?xml version="1.0" encoding="UTF-8" ?>
```

En este caso, se indica que el conjunto de caracteres que se utiliza en el documento es UTF-8, el cual permite el uso de acentos o caracteres como la ñ.

### Estándares de codificación de caracteres:

| Estándar ISO         | Código de país                   |
|----------------------|----------------------------------|
| UTF-8 (Unicode)      | Conjunto de caracteres universal |
| ISO-8859-1 (Latin-1) | Europa occidental, Latinoamérica |
| ISO-8859-2 (Latin-2) | Europa central y oriental        |
| ISO-8859-3 (Latin-3) | Sudoeste de Europa               |
| ISO-8859-4 (Latin-4) | Países Escandinavos, Bálticos    |
| ISO-8859-5           | Cirílico                         |

| Estándar ISO | Código de país              |
|--------------|-----------------------------|
| ISO-8859-6   | Árabe                       |
| ISO-8859-7   | Griego                      |
| ISO-8859-8   | Hebreo                      |
| ISO-8859-9   | Turco                       |
| ISO-8859-10  | Lapón. Nórdico,<br>esquimal |

## Autonomía del documento

La declaración de la autonomía del documento informa de si el documento necesita de otro para su interpretación. Para esto, se utiliza el atributo `standalone`:

```
<?xml version="1.0" encoding="iso-8859-1" standalone="yes" ?>
```

En este caso, se indica que el documento es independiente. De no ser así, el atributo `standalone` hubiese tomado el valor `no`.

Para declarar el atributo `standalone`, hay que definir el prólogo completo.

## Declaración del tipo de documento

Define qué tipo de documento estamos creando con la finalidad de ser procesado correctamente. Toda declaración de tipo de documento comienza por `<!DOCTYPE` seguido del nombre del tipo y el caracter `>`. Por ejemplo:

```
<!DOCTYPE html>
```

## Ejemplar

Es la **parte principal** de un **documento XML**, ya que contiene los datos reales del documento. Está formado por elementos anidados.

## Elementos

Los **elementos** son los distintos bloques de información que permiten definir la estructura de un documento XML. Por ejemplo:

```
<libro>XML práctico</libro>
```

Los elementos están delimitados por una etiqueta de **apertura** y una etiqueta de **cierre**. En el ejemplo mostrado:

- Etiqueta de apertura: `<libro>`
- Etiqueta de cierre: `</libro>`

A su vez, los elementos pueden estar formados por otros **elementos** y/o por **atributos**. Por ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE libro>
<libro>
  <titulo>XML practico</titulo>
  <autor>Sebastien Lecomte</autor>
  <autor>Thierry Boulanger</autor>
  <editorial>Ediciones Eni</editorial>
  <isbn>978-2-7460-4958-1</isbn>
  <edicion>1</edicion>
  <paginas>347</paginas>
</libro>
```

En este caso, el ejemplar es el elemento `<libro>`. A su vez, `<libro>` está compuesto de los elementos:

- `<autor>`
- `<editorial>`
- `<isbn>`
- `<edicion>`
- `<paginas>`

En realidad, el ejemplar es el **elemento raíz** (*root*) de un documento XML (en este ejemplo, el elemento `<libro>`). Todos los datos de un documento XML han de pertenecer a un elemento del mismo.

Los **nombres** de las etiquetas han de ser **autodescriptivos**, lo que facilita el trabajo que se hace con ellas.

## Sintaxis

La formación de elementos ha de cumplir ciertas normas para que queden perfectamente definidos y que el documento XML al que pertenecen pueda ser interpretado por los procesadores XML sin generar ningún error fatal.

A continuación, se describen las reglas de sintaxis del lenguaje XML.

## Único elemento raíz

En todo documento XML debe existir **un elemento raíz** (*root element*), y sólo uno.

Por ejemplo, el siguiente ejemplo sería un XML no válido (tiene dos elementos raíz):

```
<libro>XML Práctico</libro>
<cuaderno>Lenguaje de marcas</libro>
```

El hecho de que tenga un elemento raíz, permite representar los datos en forma de árbol.

Supongamos el siguiente documento XML:

```
<persona>
  <nombre>Elsa</nombre>
  <mujer/>
  <fecha-de-nacimiento>
    <día>18</día>
    <mes>6</mes>
    <año>1996</año>
  </fecha-de-nacimiento>
  <ciudad>Pamplona</ciudad>
</persona>
```

Se podría representar gráficamente de la siguiente manera:



## Etiqueta de apertura y cierre

Todos los elementos tienen una etiqueta de inicio y otra de cierre. Por ejemplo:

```
<libro>XML Práctico</libro>
```

Lo que sería incorrecto es lo siguiente (falta la etiqueta de cierre):

```
<libro>XML Práctico
```

En el caso de que en el documento existan **elementos vacíos** (*empty elements*), se pueden sustituir las etiquetas de inicio y cierre por una de elemento vacío. Ésta se construye como la etiqueta de inicio, pero sustituyendo el carácter `>` por `/>`. Por ejemplo, supongamos el siguiente elemento:

```
<libro></libro>
```

Sería equivalente a:

```
<libro/>
```

## Anidación de elementos

Al **anidar elementos** (introducir unos dentro de otros), hay que tener en cuenta que no puede cerrarse un elemento que contenga algún otro elemento que aún no se haya cerrado.

Por ejemplo, esto es incorrecto (se cierra el elemento `libro` antes de `título`):

```
<libro>
  <titulo>XML Práctico</libro>
</titulo>
```

Lo correcto es:

```
<libro>
  <titulo>XML Práctico</titulo>
</libro>
```

## Nomenclatura de etiquetas

Los nombres de las etiquetas de **inicio** y de **cierre** de un mismo elemento han de ser **idénticos, respetando las mayúsculas y minúsculas**. Por ejemplo:

```
<cuaderno></cuaderno>
```

Sin embargo, no sería válido lo siguiente:

```
<Libro></libro>
```

Las normas de sintaxis básicas en relación a los nombres de etiquetas son:

- Todos los nombres de los elementos son sensibles a letras minúsculas y mayúsculas (*case sensitive*).
- Pueden contener letras minúsculas, letras mayúsculas, números, puntos (.), guiones medios (-) y guiones bajos (\_).
- Asimismo, pueden contener el carácter dos puntos (:). Pero, su uso se reserva para definir espacios de nombres.
- El primer carácter tiene que ser una letra o un guion bajo (\_).
- Las letras no inglesas (á, Á, ñ, Ñ, etc.) están permitidas. Pero, al igual que el carácter guion medio (-) y el punto (.), se recomienda no utilizarlos para reducir posibles incompatibilidades o errores en programas que no los interpreten bien.
- No puede comenzar por la cadena xml, ni ninguna de sus versiones en que se cambien mayúsculas y minúsculas (XML, Xml, xML, etc.).

## Contenido de los elementos

No se pueden utilizar directamente los caracteres >, <, &, " y ' en el contenido de los elementos, ya que son caracteres reservados.

## Uso de comillas simples y dobles:

En el caso de las comillas simples (') y dobles ("), existe un caso donde sí se pueden utilizar: en **atributos**. Aunque respetando ciertas restricciones.

En el caso de tener que utilizar alguno de estos caracteres, se sustituyen por las siguientes cadenas:

| Caracter | Entidad | Representación |
|----------|---------|----------------|
| >        | gt      | &gt;           |
| <        | lt      | &lt;           |
| &        | amp     | &amp;          |
| "        | quot    | &quot;         |
| '        | apos    | &apos;         |

Al nombre que hace referencia a un caracter se le denomina **entidad**. Por ejemplo, la entidad `gt` hace referencia al caracter `>`.

Cuando utilizamos una entidad en un documento XML debe hacerse mediante su representación (ver tabla), es decir, el nombre de la entidad entre los símbolos `&` y `;`. Por ejemplo:

```
<libro>Elemento &lt;libro&lt;</libro>
```

El contenido del elemento `libro` sería:

```
Elemento <libro>
```

## Caracteres especiales

Para utilizar caracteres especiales (como £, ©, ®, etc.) hay que usar las expresiones `&#D;` o `&#H;`, donde `D` y `H` se corresponden respectivamente con el número decimal o hexadecimal asociado al caracter que se quiere representar en el **código UNICODE**.

Por ejemplo, para incluir €, se usarían las cadenas `&#8364;` o `&#x20AC;`.

### CARACTERES ASCII

En el siguiente enlace encontrarás una tabla con los caracteres ASCII, el nombre HTML, y el número HTML de cada uno de ellos que te será imprescindible a la hora de

realizar documentos en HTML y XML.

<http://ascii.cl/es/codigos-html.htm>

## Atributos

Los atributos permiten añadir propiedades a los elementos de un documento.

Supongamos que tenemos el siguiente elemento:

```
<precio>14.95</precio>
```

El elemento anterior indica que el precio es 14.95. Si queremos indicar que el precio está en euros, podemos añadir un atributo `divisa` de la siguiente forma:

```
<precio divisa="euro">14.95</precio>
```

A un elemento, podemos añadir uno o más atributos. Por ejemplo:

```
<precio divisa="euro" descuento="no">14.95</precio>
```

Características de los atributos son:

- No pueden organizarse en ninguna jerarquía.
- No pueden contener ningún otro elemento o atributo.
- No reflejan ninguna estructura lógica.

### USO DE ATRIBUTOS

No se debe utilizar un atributo para contener información susceptible de ser dividida.

Un ejemplo completo de elementos con atributos es el siguiente:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE biblioteca>
<biblioteca>
  <ejemplar tipo_ejem="libro" titulo="XML práctico"
editorial="Ediciones Eni">
    <tipo>
      <libro isbn="978-2-7460-4958-1" edicion="1" paginas="347">
</libro>
    </tipo>
    <autor nombre="Sebastien Lecomte"></autor>
    <autor nombre="Thierry Boulanger"></autor>
    <autor nombre="Angel Belinchon Calleja" funcion="traductor">
</autor>
    <prestado lector="Pepito Grillo">
      <fecha_pres dia="13" mes="mar" año="2009"></fecha_pres>
      <fecha_devol dia="21" mes="jun" año="2009"></fecha_devol>
    </prestado>
  </ejemplar>
</biblioteca>
```

Como se observa en el ejemplo, los atributos se definen siempre en la etiqueta de apertura (o bien de un elemento vacío).

|                                                               |                                        |
|---------------------------------------------------------------|----------------------------------------|
| <code>&lt;precio divisa="euro"&gt;14.95&lt;/precio&gt;</code> | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;precio divisa="euro" /&gt;</code>                   | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;precio&gt;14.95&lt;/precio divisa="euro"&gt;</code> | <code>&lt;!-- Incorrecto --&gt;</code> |

A continuación del nombre del elemento o de la definición de otro atributo, siempre separado de ellos por un espacio. Los valores del atributo van precedidos de `=`, seguido del valor. El valor debe definirse entre comillas simples o dobles. En cualquier caso, cuando se utiliza un tipo de comillas, éstas no pueden utilizarse dentro del contenido.

Por ejemplo, si queremos que el valor de un atributo sea `"Titanic"` (el título incluye comillas dobles), podemos definirlo de la siguiente manera:

|                                                                      |                                        |
|----------------------------------------------------------------------|----------------------------------------|
| <code>&lt;película nombre='Titanic' /&gt;</code>                     | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;película nombre="&amp;quot;Titanic&amp;quot;" /&gt;</code> | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;película nombre=""Titanic"" /&gt;</code>                   | <code>&lt;!-- Incorrecto --&gt;</code> |

Si queremos que el valor de un atributo sea `'Titanic'` (el título incluye comillas simples), podemos definirlo de la siguiente manera:

|                                                                      |                                        |
|----------------------------------------------------------------------|----------------------------------------|
| <code>&lt;película nombre=""Titanic"" /&gt;</code>                   | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;película nombre='&amp;apos;Titanic&amp;apos;' /&gt;</code> | <code>&lt;!-- Correcto --&gt;</code>   |
| <code>&lt;película nombre=''Titanic'' /&gt;</code>                   | <code>&lt;!-- Incorrecto --&gt;</code> |

Recuerda que:

- `&quot;` es la entidad para representar las comillas dobles (").
- `&apos;` es la entidad para representar las comillas simples (').

Aun así, muchos navegadores representan los documentos XML empleando comillas dobles independientemente de qué tipo de comilla se ha incluido.

Los nombres de los atributos han de cumplir las mismas reglas que los de los elementos.

## Espacios en blanco

En un documento XML, los espacios en blanco, las tabulaciones y los retornos de carro (salto de línea) pueden ser tratados de un modo especial.

## Interpretado de los espacios en blanco

En este apartado, se analizará el comportamiento predeterminado de los espacios en blanco, tabulaciones y retornos de carro en las diferentes ítems de XML.

## Contenido

Varios espacios en blanco en el **contenido** de un elemento, se interpretan como uno solo.

Por ejemplo, los dos siguientes documentos se interpretarían de la misma forma:

```
<?xml version="1.0" encoding="UTF-8"?>
<películas>
  <película>El discurso del rey</película>
  <película>En      tierra      hostil</película>
  <película>Una
    mente
maravillosa</película>
</películas>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<peliculas>
  <pelicula>El discurso del rey</pelicula>
  <pelicula>En tierra hostil</pelicula>
  <pelicula>Una mente maravillosa</pelicula>
</peliculas>
```

## Atributos

De igual modo ocurre con los valores de los atributos.

Por ejemplo, los dos siguientes documentos se interpretarían de la misma forma:

```
<?xml version="1.0" encoding="UTF-8"?>
<series>
  <serie numeros="2 4 6 8"/>
  <serie numeros="3
6
9
12 15"/>
</series>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<series>
  <serie numeros="2 4 6 8"/>
  <serie numeros="3 6 9 12 15"/>
</series>
```

## Elementos

Los espacios en blanco entre elementos no se tienen en cuenta.

Por ejemplo, los tres siguientes documentos se interpretarían de la misma forma:

```
<?xml version="1.0" encoding="UTF-8"?>
<datos>
  <dato>1</dato>
  <dato>2</dato>
  <dato>3</dato>
</datos>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<datos><dato>1</dato><dato>2</dato><dato>3</dato></datos>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<datos><dato>1</dato>    <dato>2</dato>

<dato>3</dato></datos>
```

## xml:space

Para modificar el interpretado de los espacios en blanco, tabulaciones y retornos de carro en un documento XML, se puede utilizar el atributo predefinido `xml:space` con el valor `preserve`. De esta forma, se le indica al intérprete que los espacios en blanco en el contenido de dicho elemento (y de sus hijos) deben ser conservados.

```
<clasificacion xml:space="preserve">
1      Fernando Alonso      1:55.341
2      Lewis Hamilton       1:55.729
3      Sebastian Vettel     1:56.122
</clasificacion>
```

Los valores que acepta el atributo `xml:space` son:

- `preserve`
- `default`

El valor `default` indica al intérprete que es él quien decide cómo tratar los espacios en blanco.





Aún indicando el valor `preserve`, hay que tener en cuenta que no todos los intérpretes reconocen este atributo. Es el caso de Mozilla Firefox y Google Chrome.

## Documentos XML bien formados

Un documento bien formado es aquel que cumple las reglas sintáticas que define la recomendación del W3C para el estándar XML.

### Herramienta de validación

La W3C ofrece un [validador online](#) de documentos XML bien formados. Es de utilidad para verificar si un documento está bien formado.

Supongamos el siguiente documento:

```
<?xml version="1.0"?>
<mensaje>
  <destinatario>Tomas</ destinatario>
  <remitente>Juan</ remitente>
  <asunto>
    <contenido> No olvides ir a recogerme al aeropuerto mañana por la
mañana!</contenido>
  </mensaje>
```

No es válido por los siguientes motivos:

- Las etiquetas de cierre `</ destinatario>` y `</ remitente>` contienen espacios en su nombre. Los espacios no son válidos.
- El elemento `<asunto>` no está cerrado.

### Validator online

Para el documento anterior, el validador online de la W3C nos mostraría el siguiente mensaje:

Como se puede observar, indica que hay un error en la tercera línea, pero no nos especifica cuál es el error. Es labor del programador identificarlo.

Como alternativa, se puede obtener información más significativa empleando **validadores más avanzados** como **XML Validation**.

La forma válida del documento XML sería la siguiente:

```
<?xml version="1.0"?>
<mensaje>
  <destinatario>Tomas</destinatario>
  <remitente>Juan</remitente>
  <asunto/>
  <contenido> No olvides ir a recogerme al aeropuerto mañana por la
mañana!</contenido>
</mensaje>
```

## Espacios de nombres

Los espacios de nombres (*namespaces*) permiten resolver las **ambigüedades** que se pueden producir al juntar dos documentos XML distintos que han utilizado el mismo nombre de etiqueta para representar cosas distintas. Los *namespaces* definen la **pertenencia** de los elementos y los atributos de un documento XML a un **contexto** de un vocabulario XML.

Supongamos los siguientes documentos (`baraja.xml` y `restaurante.xml`):

baraja.xml

```
<carta>
  <palo>Corazones</palo>
  <numero>7</numero>
</carta>
```

restaurante.xml

```
<carta>
  <carnes>
    <filete_de_tenera precio="12.95"/>
    <solomillo_a_la_pimienta precio="13.60"/>
  </carnes>
  <pescados>
    <lenguado_al_horno precio="16.20"/>
    <merluza_en_salsa_verde precio="15.85"/>
  </pescados>
</carta>
```

Si juntamos ambos documentos en un solo, quedaría de la siguiente forma:

```
<ejemplo> <!-- Debemos añadir un elemento raíz para poder juntarlos -
->
  <carta>
    <palo>Corazones</palo>
    <numero>7</numero>
  </carta>
  <carta>
    <carnes>
      <filete_de_ternera precio="12.95"/>
      <solomillo_a_la_pimienta precio="13.60"/>
    </carnes>
    <pescados>
      <lenguado_al_horno precio="16.20"/>
      <merluza_en_salsa_verde precio="15.85"/>
    </pescados>
  </carta>
</ejemplo>
```

Como se puede observar, al fusionar los dos documentos en uno, tenemos un elemento `<carta>` que tiene varios significados dentro del documento. Esto **no implica** que el **documento XML no sea válido**, sino que a **nivel semántico** tenemos un elemento que significa cosas diferentes.

Si se incluyen ambos elementos `<carta>` en un documento XML, se origina un conflicto de nombres. Para resolverlo, se pueden utilizar espacios de nombres. Por ejemplo, escribiendo:

```
<?xml version="1.0" encoding="UTF-8"?>
<ejemplo xmlns:e1="https://www.lmsgi.com/baraja"
xmlns:e2="https://www.lmsgi.com/restaurante">
  <e1:carta>
    <e1:palo>Corazones</e1:palo>
    <e1:numero>7</e1:numero>
  </e1:carta>
  <e2:carta>
    <e2:carnes>
      <e2:filete_de_ternera precio="12.95"/>
      <e2:solomillo_a_la_pimienta precio="13.60"/>
    </e2:carnes>
    <e2:pescados>
      <e2:lenguado_al_horno precio="16.20"/>
      <e2:merluza_en_salsa_verde precio="15.85"/>
    </e2:pescados>
  </e2:carta>
</ejemplo>
```

Los espacios de nombres permiten dar un nombre único a cada elemento, indexándolos según el nombre del vocabulario adecuado. Además están asociados a un **URI** (*Uniform Resource Identifier*) que los identifica de forma única.

En el documento, las etiquetas ambiguas se sustituyen por otras en las que el nombre del elemento está precedido de un **prefijo**, el cual determina el contexto al que pertenece la etiqueta. El prefijo debe ir separado del nombre de la etiqueta por dos puntos (:).

Por ejemplo, supongamos que tenemos el siguiente elemento:

```
<palo>Corazones</palo>
```

Si quisiésemos añadir un prefijo **e1**, se indicaría de la siguiente manera:

```
<e1:palo>Corazones</e1:palo>
```

Esta etiqueta se denomina **nombre cualificado**. Al definir el prefijo, hay que tener en cuenta que:

- No puede contener espacios ni caracteres especiales.
- No puede comenzar por un dígito.

Antes de poder utilizar un prefijo de un *namespace*, para resolver la ambigüedad de dos o más etiquetas, es necesario **declarar el espacio de nombres**, es decir, asociar un índice con

el URI asignado al espacio de nombres. Esto se realiza mediante un atributo especial `xmlns`.

La declaración debe hacerse entre el prólogo y el ejemplar de un documento XML. Por norma general, lo que se utiliza es una URL (un enlace a una web).

Los espacios de nombres pueden definirse en el elemento raíz o, directamente, en los elementos que los vayan a utilizar, aunque lo más habitual es definirlos en el elemento raíz. Por ejemplo, los siguientes documentos definen correctamente el *namespace*:

```
<?xml version="1.0" encoding="UTF-8"?>
<ejemplo xmlns:e1="https://www.lmsgi.com/baraja">
  <e1:carta>
    <e1:palo>Corazones</e1:palo>
    <e1:numero>7</e1:numero>
  </e1:carta>
</ejemplo>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<ejemplo>
  <e1:carta xmlns:e1="https://www.lmsgi.com/baraja">
    <e1:palo>Corazones</e1:palo>
    <e1:numero>7</e1:numero>
  </e1:carta>
</ejemplo>
```

## CDATA

Un documento XML puede contener secciones CDATA (*Character DATA*) para escribir texto que **no** se desea que sea **analizado**. Por ejemplo, esto puede ser útil cuando se quiere escribir texto que contenga alguno de los caracteres problemáticos: `<` o `&`.

En un documento XML, para incluir una sección CDATA, esta se escribe comenzando con la cadena de caracteres `<![CDATA[` y terminando con los caracteres `]]>`.

```
<?xml version="1.0" encoding="UTF-8"?>
<ejemplo_CDATA>
<![CDATA[
#include <stdio.h>
int main() {
    float nota;
    printf( "\n  Introduzca nota (real): " );
    scanf( "%f", &nota );
    if ( 5 <= nota )
        printf( "\n  APROBADO" );
    return 0;
}
]]>
</ejemplo_CDATA>
```

Dentro de una sección CDATA no se puede escribir la cadena `]]>`. En consecuencia, no se pueden anidar secciones CDATA, es decir, introducir secciones CDATA dentro de otras.

Por otra parte, no está permitido escribir espacios en blanco o saltos de línea en las cadenas de inicio `<![CDATA[` o fin `]]>` de una sección CDATA.